

Track: Edge Digital Twin Architecture for Continuous Physiological Monitoring

Domain: Healthcare AI, Biomedical Signal Processing, Edge Computing, Digital Twins, TinyML, IoT

Contents

SECTION	TOPIC
1	Introduction and system overview
2	Materials, tools, and software requirements
3	Safety, workstation, and handling
4	AFE4950EVM software and firmware preparation
5	Sensor and hardware connections
6	TI GUI configuration and capture
7	Recorded dataset and export verification
8	Python project setup
9	Signal analysis and output interpretation
10	CardioTwin dashboard operation
11	SK-AM62B processor bring-up
12	USB DFU/U-Boot validation
13	Processor checks and AFE4950-to-SK-AM62B USB connection
Appendix A	Complete command reference
Appendix B	Troubleshooting notes
Appendix C	File inventory and reference documents

1 Introduction and system overview

1.1 Purpose

CardioTwin is an instructional physiological monitoring prototype built around the Texas Instruments AFE4950 evaluation module and the SK-AM62B processor platform. The AFE4950 acquires ECG and optical-channel measurements. The recorded signals are exported to a Windows laptop, processed in Python, and replayed in a Streamlit-based monitoring dashboard. Separately, the SK-AM62B was powered, enumerated, and operated through its U-Boot console as the intended edge-processor platform.

1.2 Demonstrated signal path

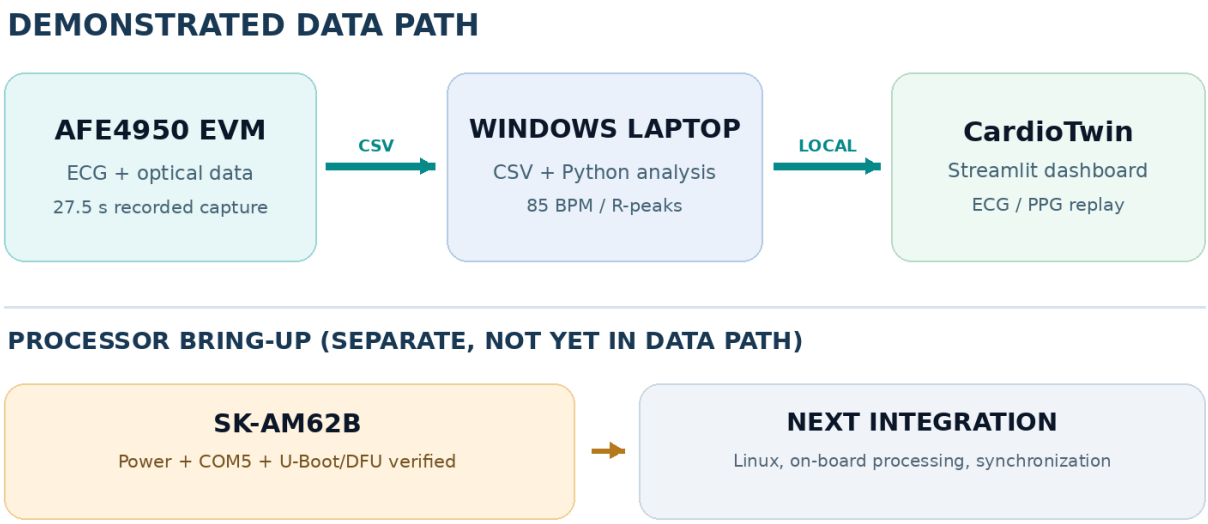


Figure 3. System architecture: demonstrated laptop-hosted data path and separately verified processor console.

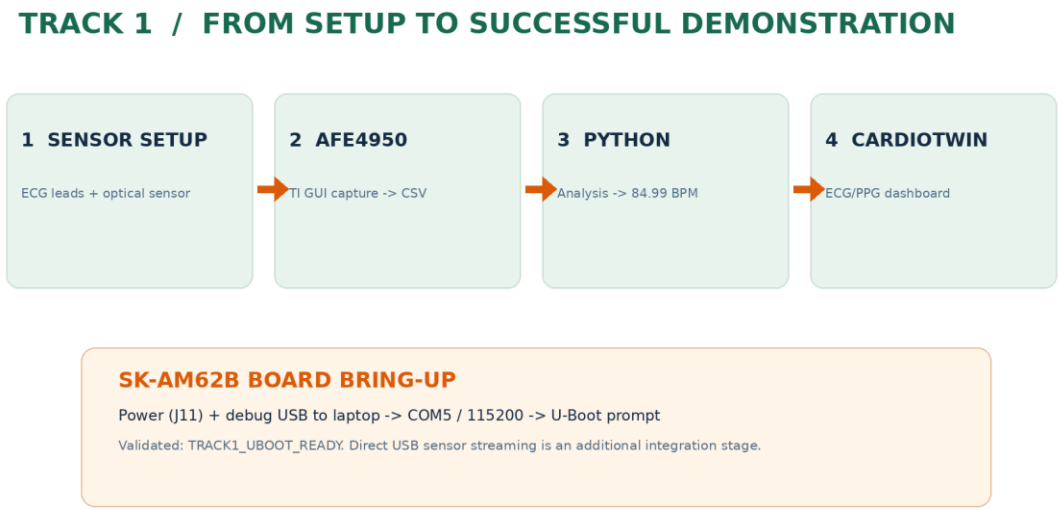


Figure 4. End-to-end reference demonstration: AFE acquisition through the working dashboard, plus the processor bring-up checkpoint.

1.3 Functional objectives

- Connect and initialize the AFE4950EVM with its supported TI software.
- Acquire ECG and optical PPG data at the mixed-mode sampling rates.

- Export the measurement into a timestamped CSV and preserve the original file.
- Calculate ECG beat candidates and estimate heart rate from the recorded segment.
- Inspect both channels with filtering, R-peak indicators, and a recording-navigation control.
- Start and interact with the SK-AM62B U-Boot console over a serial debug connection.
- Understand the physical board-to-board USB cable and the verification needed for processor-side acquisition.

1.4 Verified reference results

MEASUREMENT	OBSERVED VALUE
Recording length	27.48 s (displayed as 27.5 s)
ECG sample rate	500 Hz
Optical sampling rate	50 Hz
ECG samples	13,740
Optical samples	1,374
ECG beat candidates	39
Optical peak candidates	37
ECG/optical candidate matches	21
Reported heart rate	84.99 BPM (displayed as 85 BPM)

2 Materials, tools, and software requirements

2.1 Hardware inventory

ITEM	USE
AFE4950EVM	ECG and optical biosignal analog front-end; USB interface to TI GUI
Optical sensor board + 10-pin cable	PPG acquisition through AFE4950 J1 sensor connection
ECG cable set and electrodes / approved simulator	ECG input connections to AFE4950
TI SK-AM62B starter kit	Target embedded processor and U-Boot serial-console tests
USB-C PD power adapter	Power for SK-AM62B through designated J11 USB-C input
USB-A-to-Micro-USB data cable + debug USB	Two-board physical hookup and SK-AM62B serial console
Windows laptop	TI software, VS Code, analysis, and dashboard

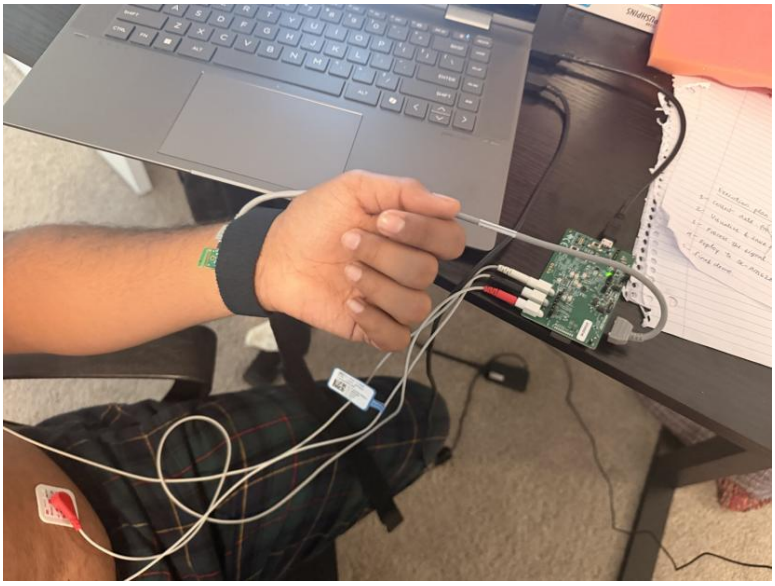


Figure 5. AFE4950 on the workbench with ECG leads, optical sensor strap, and PC connection.

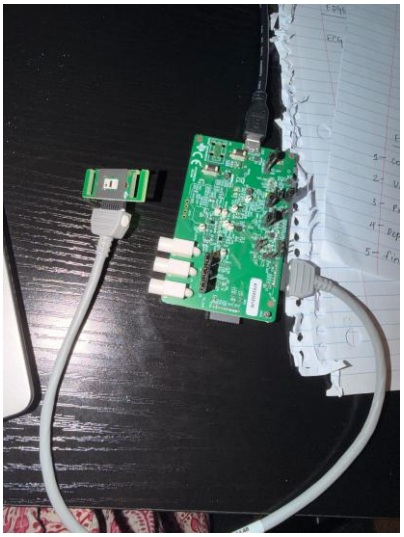


Figure 6. AFE4950 sensor module and board photographed during setup.

2.2 Software inventory

SOFTWARE	PURPOSE
TI Bio-Sensing + AFE4950 project package	AFE initialization, register setup, and signal capture
TI firmware loader / USB driver	Firmware upgrade and CDC-based USB detection when required
VS Code + Python 3 + venv	Editable local analysis workspace
numpy, pandas, scipy, plotly, streamlit, matplotlib, pyserial	Signal processing, plots, dashboard, and serial-terminal utility
dfu-util 0.11 + TI uboot-flash-writer repository	SK-AM62B USB DFU bootloader experiments

2.3 Two-board connection at a glance

There are two supported lab activities. For the recorded-data demonstration, the AFE4950 uses its micro-USB cable to communicate with the Windows laptop. For the board-to-board physical hookup, the same AFE4950 micro-USB connector can instead be plugged into the SK-AM62B USB-A host port with a suitable USB-A-to-Micro-USB DATA cable. Only one USB host can own this AFE cable at a time; unplug it from the PC before connecting it to the SK-AM62B.

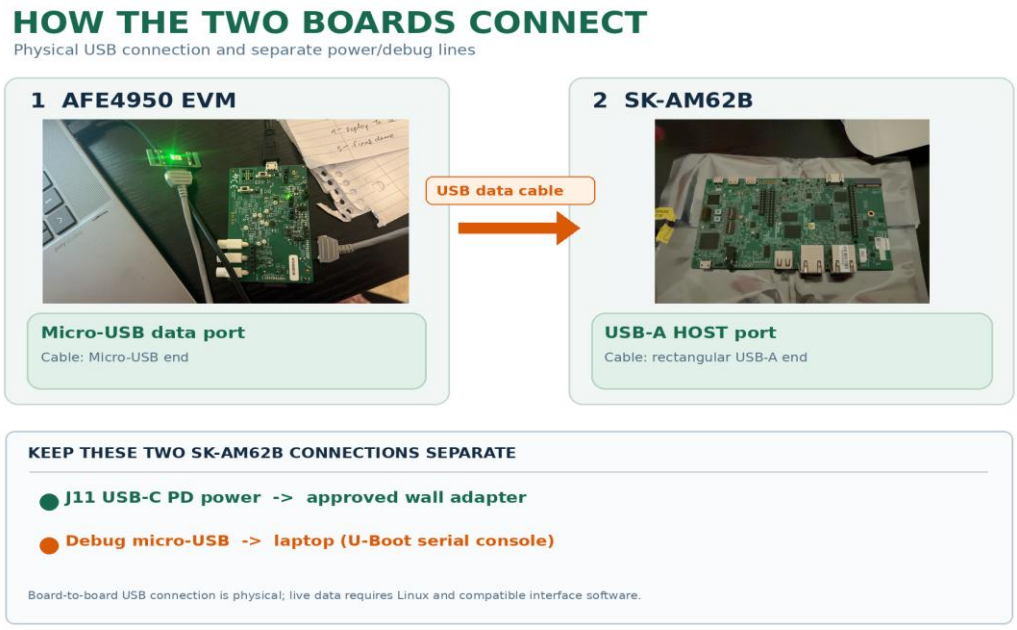


Figure 7. Board-to-board wiring: Micro-USB on AFE4950 to USB-A host on SK-AM62B; separate J11 power and debug USB.

CABLE / PART	CONNECT FROM	CONNECT TO
USB-A-to-Micro-USB data cable	AFE4950EVM micro-USB	SK-AM62B USB-A HOST
USB-C PD power adapter	Wall power supply	SK-AM62B J11 power-only USB-C
Micro-USB debug cable	SK-AM62B debug connector	Windows laptop
10-pin optical sensor cable	PPG sensor board	AFE4950EVM J1
Color-coded ECG leads	Approved simulator / electrodes	AFE4950EVM ECG inputs

READ BEFORE CONNECTING A physical USB cable does not by itself transfer recognized ECG/PPG samples into the SK-AM62B application. Direct capture additionally needs a running Linux USB host and compatible software for the TI EVM interface. In our captured demonstration, the successful sensor-processing path used the Windows laptop.

3 Safety and workstation preparation

STEP 01 Prepare the ESD-protected work area.

Use a stable dry workbench and grounded ESD protection appropriate for the lab. Keep liquids away from open PCBs.

STEP 02 Identify the board power and data ports before attaching cables.

Power the SK-AM62B from its own approved adapter. Avoid using an unverified laptop USB-C power/data arrangement.

STEP 03 Inspect the leads and connectors.

Check electrode leads, optical sensor ribbon, and USB data cables for physical damage. Never force polarized connectors.

STEP 04 Use the privacy-masked electrode photograph only as a reference.

The photo documents the previous lab activity; it does not replace the EVM electrical-safety requirements or an approved lead-placement protocol.

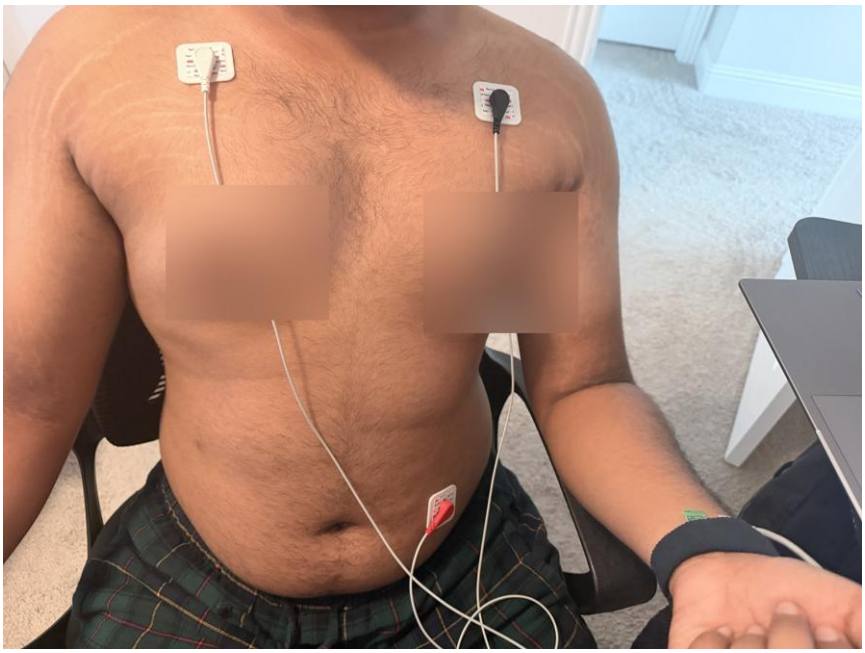


Figure 8. Reference positioning of upper-torso ECG leads, lower reference lead, and wrist optical sensor; chest areas privacy-masked.

4 AFE4950EVM software and firmware preparation

4.1 Install the TI GUI

STEP 05 Obtain the AFE4950EVM software package supplied for the lab.

The TI source guide identifies the common Bio-Sensing installer followed by the AFE4950EVM-specific installer. Use the approved lab-provided installation files.

STEP 06 Install the common TI Bio-Sensing framework.

Run the Bio-Sensing installer. The TI reference guide lists C:\Program Files\Texas Instruments\Bio-Sensing as the typical application location.

STEP 07 Install the AFE4950 project package.

Run the AFE4950EVM-specific installer, then open Bio-Sensing.exe.

STEP 08 Connect AFE4950EVM to the computer through the board micro-USB connector.

Windows should enumerate the EVM communication interface. The TI documentation describes a CDC-based USB link; install the included USB driver only if recognition fails.

STEP 09 Verify or update the AFE4950 firmware using the TI loader.

During our preparation, the firmware loader detected the device and completed a firmware programming/verification sequence. Use the TI-approved firmware version for the exact board.

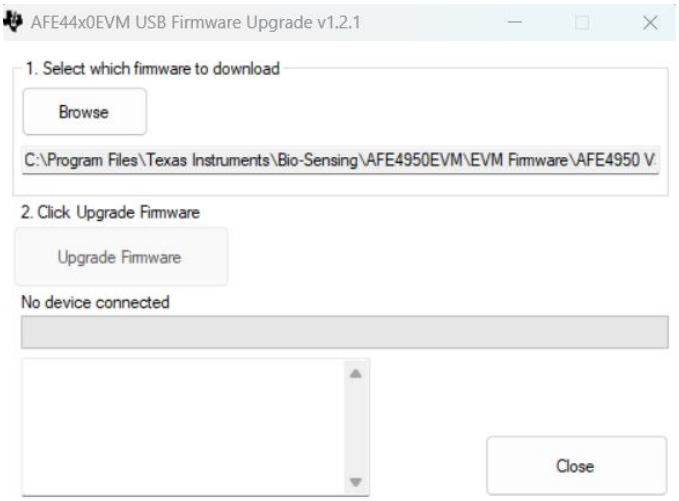


Figure 9. Firmware loader window during the USB firmware preparation.

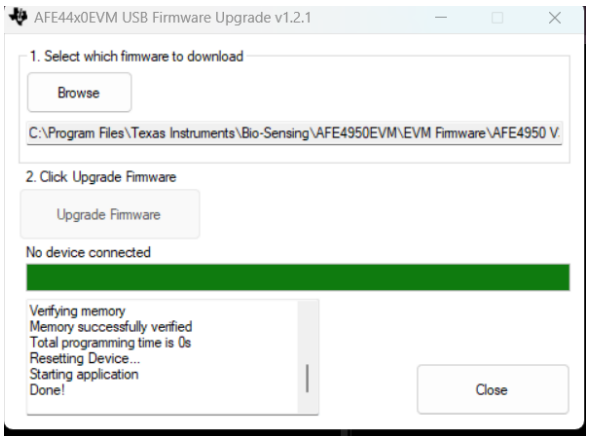


Figure 10. Firmware loader verification/completion result used to confirm the AFE interface was ready.

4.2 Firmware loader details

The TI guide describes loading firmware through BSL_USB_GUI.exe, selecting the matching firmware text file, holding the S1 bootloader switch during USB reconnection, and confirming the programming-complete message. S2 is the MCU reset switch. Record the approved version number in the lab notebook rather than guessing from a screenshot.

5 Sensor and hardware connections

5.1 Optical sensor path

STEP 10 Attach the 10-pin optical sensor lead to AFE4950 connector J1.

Match the connector orientation and the white-dot / pin-10 indicator described in the TI manual. Do not invert the cable.

STEP 11 Secure the optical sensor module using the wrist strap.

Place the sensor face against the wrist as shown in the bench photograph. Prevent strain on the sensor-board connection.



Figure 11. AFE4950 optical sensor board connection and 10-pin lead alignment.

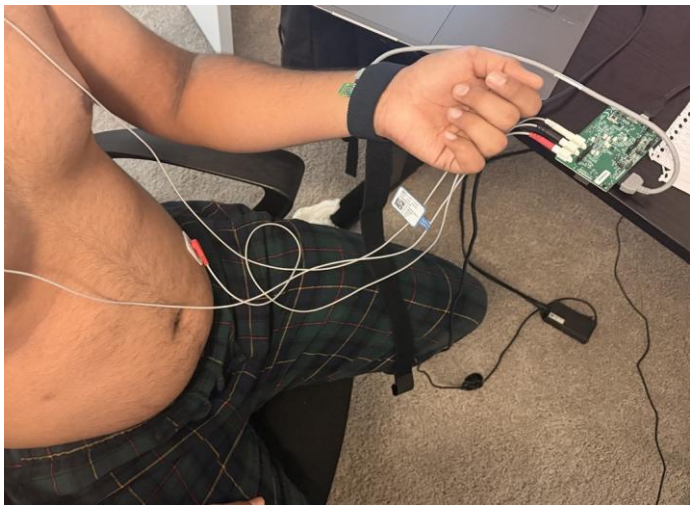


Figure 12. Wrist-mounted optical sensor connected to the AFE4950EVM.

5.2 ECG input path

STEP 12 Plug each ECG lead into the corresponding color-coded AFE4950 connector.

Use the board labels and TI guide to identify the ECG leads and their matching input roles. In a teaching lab, connect these to the approved simulator first.

STEP 13 Confirm that the AFE4950 USB data/power connection remains stable.

Verify the board status LED is illuminated and that the GUI still recognizes the device.

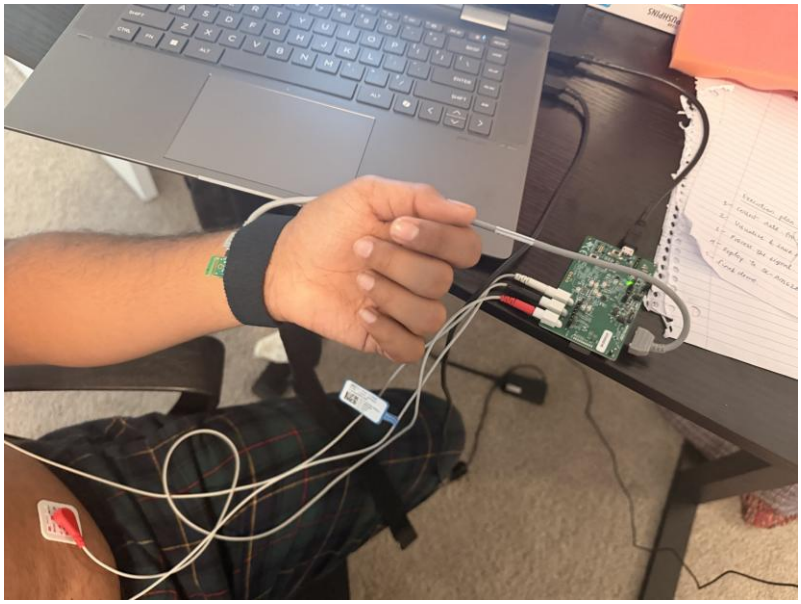


Figure 13. ECG lead cables plugged into the AFE4950EVM during the reference setup.

5.3 Connection checklist

CONNECTION	EXPECTED RESULT
PPG module to J1	10-pin sensor firmly aligned; no bending or twisting
ECG leads to AFE inputs	Lead connectors seated in the correct labeled sockets
AFE4950 micro-USB to Windows laptop	Board powered and accessible from TI Bio-Sensing application
SK-AM62B power/debug	External J11 power adapter and separate laptop debug USB

6 TI GUI configuration and data capture

6.1 Start the application and initialize the EVM

STEP 14 Launch Bio-Sensing.exe and expand the AFE4950EVM project tree.

Open its GUI or initialization script. The TI guide describes running devInit.py with F5 to load default settings.

STEP 15 Open the GUI GLOBAL SETTINGS and CAPTURE tabs.

Verify the selected mixed acquisition mode supports ECG and optical channels. The device default discussed in TI SLAU832 uses 500 Hz ECG and 50 Hz PPG.

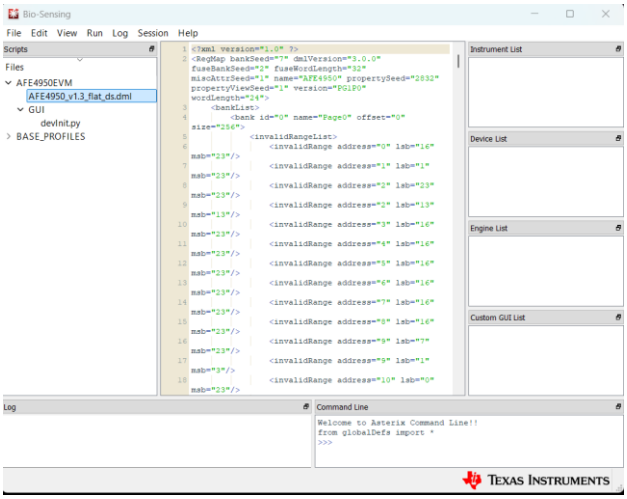


Figure 14. TI Bio-Sensing project scripts during initialization.

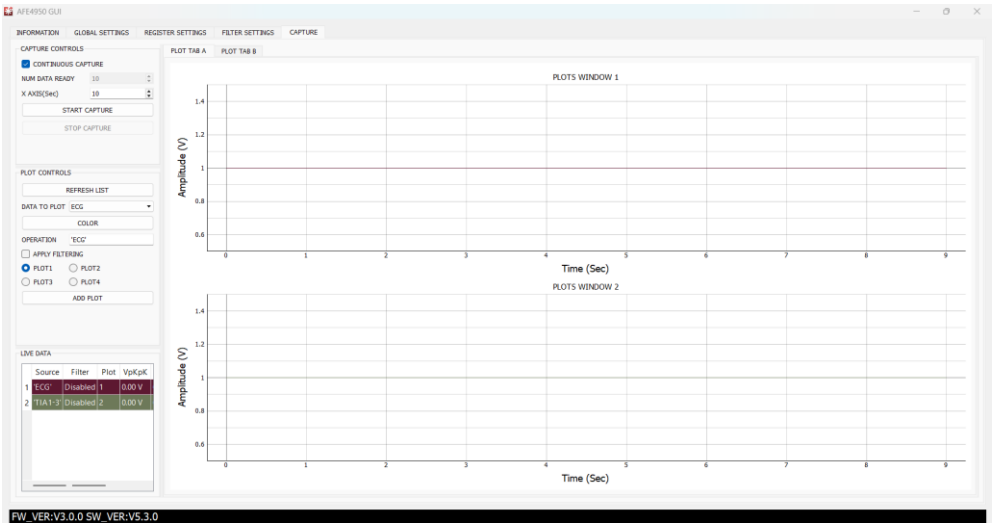


Figure 15. TI Capture workspace showing the dual-plot display and acquisition controls.

6.2 Register and channel configuration

STEP 16 Load or verify the AFE register configuration.

In GLOBAL SETTINGS choose IMPORT for the appropriate AFE4950_Settings.cfg (if supplied). Wait for a successful import indication.

STEP 17 Select the ECG channel and optical channel for plotting.

Choose the ECG signal and the optical TIA channel from the capture plot controls; route them to separate plots so both can be observed.

STEP 18 Confirm the waveform plots respond during capture.

If the plots are empty, check the selected channels, the script initialization, and the GUI capture state before changing hardware wiring.

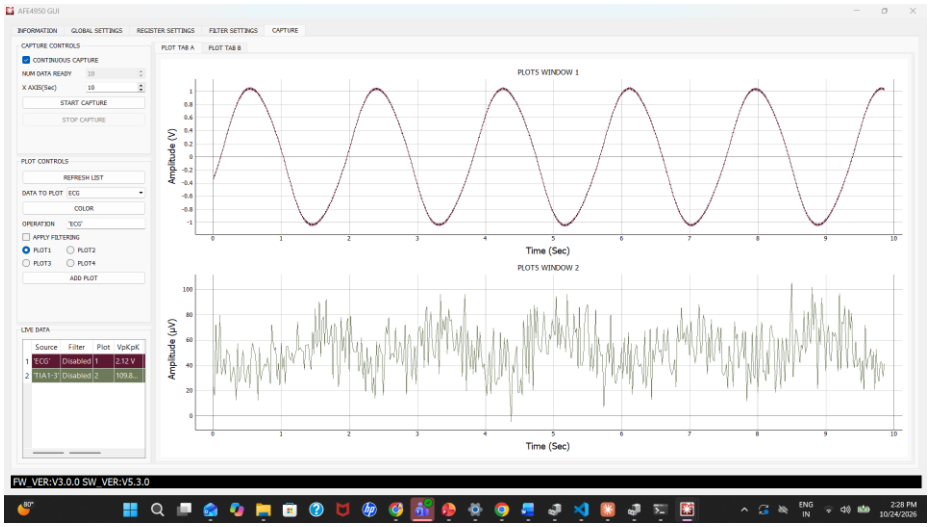


Figure 16. Example dual-channel signal plotting visible in the TI capture application.

6.3 Capture and export

STEP 19 Use Start Capture to record ECG and optical samples.

The recorded session used approximately 27.5 seconds of measurements.

STEP 20 Stop capture and inspect the plotted interval.

Confirm both selected channels contain timestamped samples before export.

STEP 21 Open GLOBAL SETTINGS > EXPORT and export CAPTURED DATA.

Save the recorded stream as AFE4950_CAPTURED_DATA.csv. Keep the source file unchanged for reproducibility.

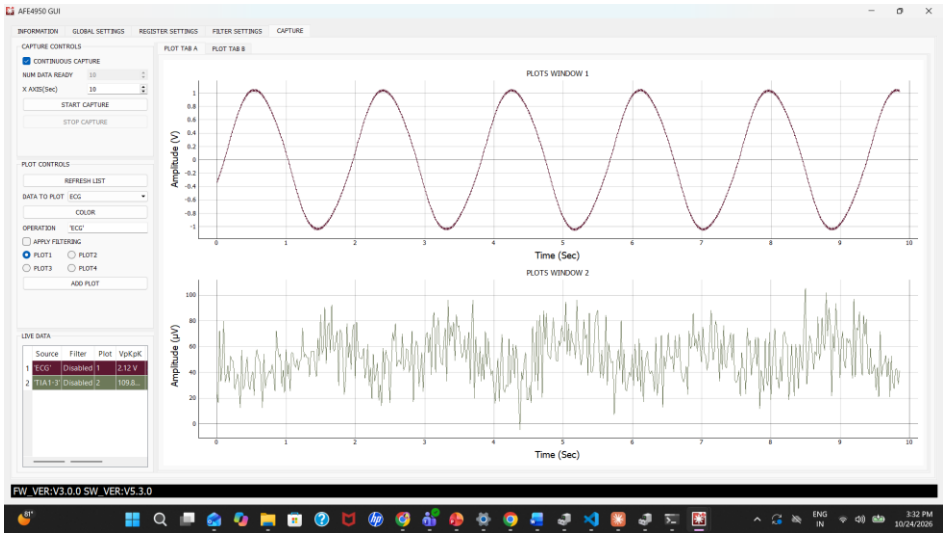
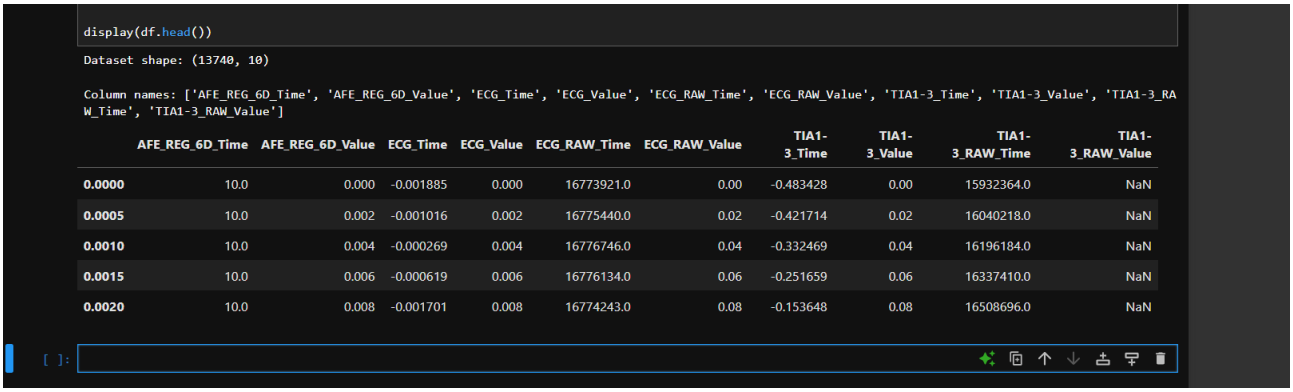


Figure 17. Waveforms shown during a TI GUI capture session.



```
display(df.head())
```

```
Dataset shape: (13740, 10)

Column names: ['AFE_REG_6D_Time', 'AFE_REG_6D_Value', 'ECG_Time', 'ECG_Value', 'ECG_RAW_Time', 'ECG_RAW_Value', 'TIA1-3_Time', 'TIA1-3_Value', 'TIA1-3_RAW_Time', 'TIA1-3_RAW_Value']
```

	AFE_REG_6D_Time	AFE_REG_6D_Value	ECG_Time	ECG_Value	ECG_RAW_Time	ECG_RAW_Value	TIA1-3_Time	TIA1-3_Value	TIA1-3_RAW_Time	TIA1-3_RAW_Value
0.0000	10.0	0.000	-0.001885	0.000	16773921.0	0.00	-0.483428	0.00	15932364.0	NaN
0.0005	10.0	0.002	-0.001016	0.002	16775440.0	0.02	-0.421714	0.02	16040218.0	NaN
0.0010	10.0	0.004	-0.000269	0.004	16776746.0	0.04	-0.332469	0.04	16196184.0	NaN
0.0015	10.0	0.006	-0.000619	0.006	16776134.0	0.06	-0.251659	0.06	16337410.0	NaN
0.0020	10.0	0.008	-0.001701	0.008	16774243.0	0.08	-0.153648	0.08	16508696.0	NaN

Figure 18. Exported timestamped AFE4950 values inspected outside the acquisition GUI.

7 Recorded dataset and export verification

7.1 Source file and columns

Our final demonstration uses a single recorded file, AFE4950_CAPTURED_DATA.csv, containing ECG and optical samples recorded at different output rates. The CSV header includes AFE_REG_6D_Time, AFE_REG_6D_Value, ECG_Time, ECG_Value, ECG_RAW_Time, ECG_RAW_Value, TIA1-3_Time, TIA1-3_Value, TIA1-3_RAW_Time, and TIA1-3_RAW_Value.

STEP 22 Copy the exported CSV into Track1_Digital_Twin\data.

Use the same original file for the analysis script and dashboard, rather than taking a new measurement for each run.

STEP 23 Verify that the CSV contains timestamped ECG and optical channels.

The reference file has 13,740 ECG time points and 1,374 TIA1-3 time points, from approximately 0 to 27.48 seconds.

STEP 24 Check the CSV delimiter behavior when loading with pandas.

The export contains a trailing delimiter on its data rows. Explicitly set index_col=False to prevent pandas from silently treating the first field as a row index.

```
from pathlib import Path
import pandas as pd
source = Path('data/AFE4950_CAPTURED_DATA.csv')
df = pd.read_csv(source, index_col=False)
print(df[['ECG_Time','ECG_Value','TIA1-3_Time','TIA1-3_Value']].head())
```

8 Python project and virtual environment

8.1 Create the workspace

STEP 25 Open VS Code and create the project directory.

The reference workstation used C:\Users\potai\Track1_Digital_Twin as its local project root.

```
mkdir Track1_Digital_Twin
cd Track1_Digital_Twin
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

STEP 26 Install the Python packages needed by the analysis and UI.

Use the virtual environment, not a different global Python instance.

```
python -m pip install numpy pandas scipy matplotlib plotly streamlit pyserial
```

STEP 27 Place the source files and recorded CSV in the project.

Keep analyze_signals.py and dashboard.py in the project root. Store the CSV and generated measurements under data.

```
Track1_Digital_Twin/
  .venv/
  analyze_signals.py
  dashboard.py
  data/
    AFE4950_CAPTURED_DATA.csv
    metrics.json
    candidate_matches.csv
```

STEP 28 Read the CSV relative to the script location.

Use Path(__file__).resolve().parent / "data" / "AFE4950_CAPTURED_DATA.csv" to avoid a working-directory error.

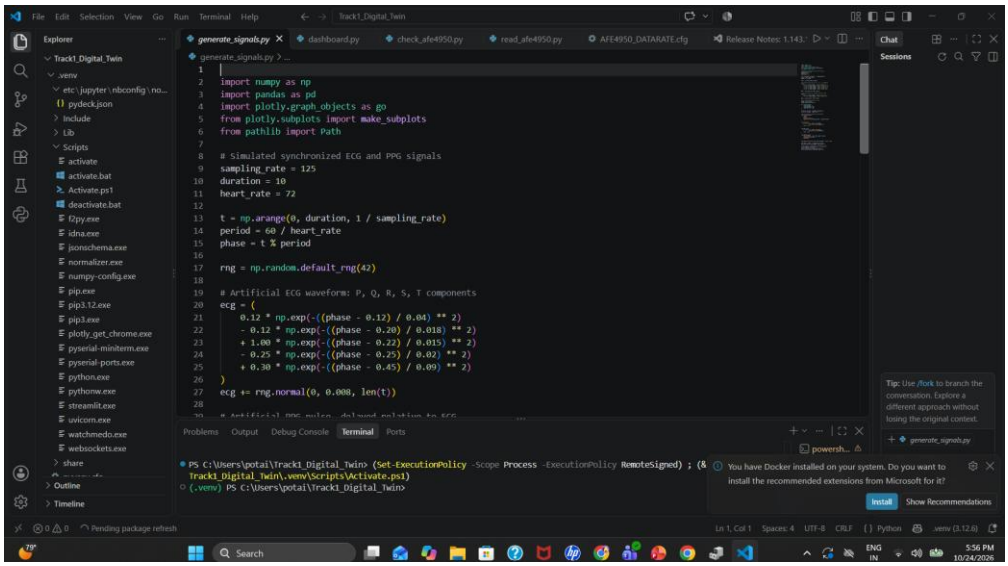


Figure 19. Python and VS Code development workspace during the prototype build.

9 ECG/PPG processing and output interpretation

9.1 Analysis workflow

The analysis program loads the acquired channels, applies signal preprocessing, detects candidate ECG R-peaks and optical peaks, pairs candidates within its configured timing rules, computes a recording-level heart-rate estimate, and writes machine-readable outputs for the dashboard.

STEP 29 Activate the virtual environment in the VS Code PowerShell terminal.

The prompt should begin with (.venv) and the current directory should be Track1_Digital_Twin.

STEP 30 Run the reference analysis file.

This reads the AFE4950 CSV and writes the processed metrics.

```
python analyze_signals.py
```

STEP 31 Verify the terminal output.

The completed analysis displayed 39 ECG beat candidates, 37 optical peak candidates, 21 candidate ECG/PPG matches, and a heart-rate estimate of 84.99 BPM.

```
TRACK 1 - REAL AFE4950 ANALYSIS
ECG beats: 39
Candidate PPG peaks: 37
ECG-PPG candidate matches: 21
Heart rate: 84.99 BPM
Saved to: C:\Users\potai\Track1_Digital_Twin\data
```

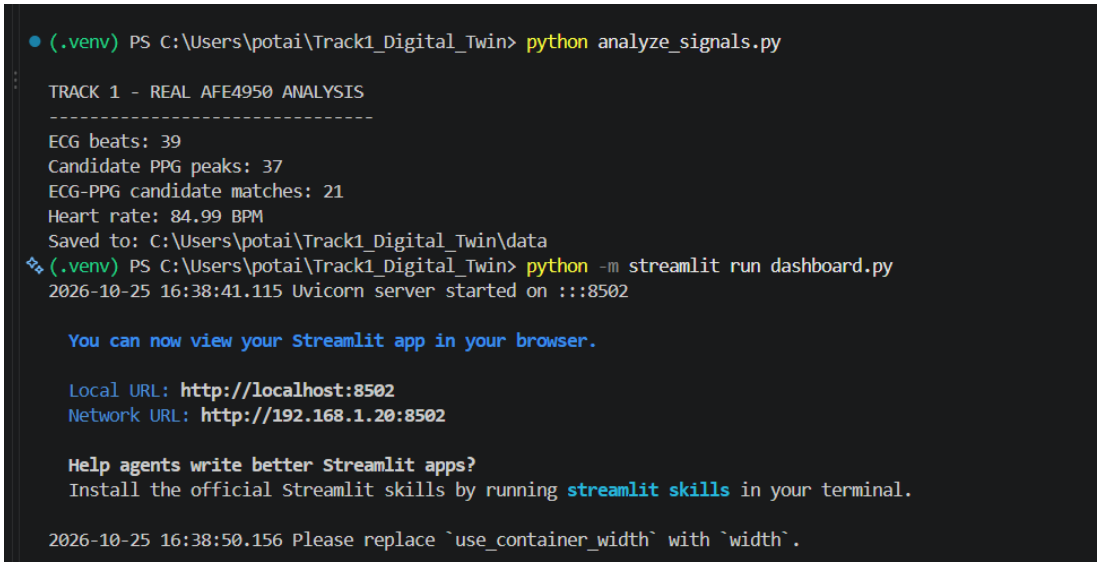


Figure 20. Recorded Python console results and Streamlit server start.

9.2 Interpretation notes

- R-peaks are beat candidates selected from the ECG waveform by the prototype algorithm.
- Optical peak candidates and ECG/PPG matches describe internal matching events; they are not validated pulse transit time or clinical blood-pressure readings.
- The heart-rate display rounds the recorded estimate to 85 BPM.

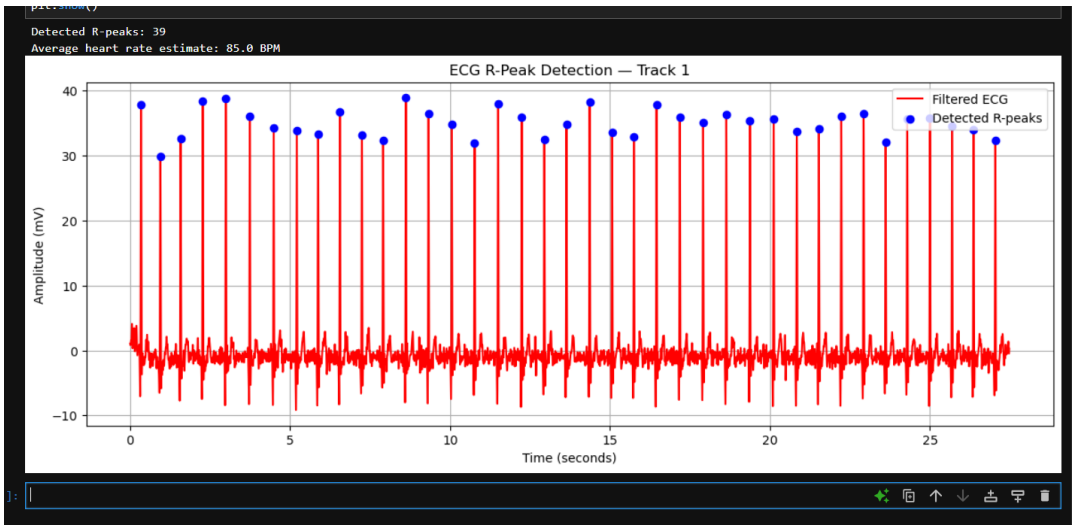


Figure 21. Earlier analysis view showing identified ECG waveform peaks.

10 CardioTwin dashboard setup and operation

10.1 Start the application

STEP 32 Open a second VS Code terminal with the virtual environment active.

Keep the analysis outputs in the same data folder.

STEP 33 Start the Streamlit interface.

Streamlit assigns an available port; in the documented session the dashboard opened on localhost:8502.

```
python -m streamlit run dashboard.py
```

STEP 34 Open the local address displayed by Streamlit.

The reference terminal displayed http://localhost:8502.



Figure 22. CardioTwin cardiovascular overview, 85 BPM, recording duration, sample rates, and controls.

10.2 Interactive monitoring tools

STEP 35 Review the Cardiovascular Overview cards.

Confirm heart rate ~85 BPM, capture duration ~27.5 seconds, ECG 500 Hz, and PPG 50 Hz.

STEP 36 Use the Viewing Window slider.

Increase or reduce the displayed time range without changing the stored CSV.

STEP 37 Move the Navigate Recording slider.

Select earlier and later time windows within the 27.5-second captured session.

STEP 38 Toggle Digital Filtering and R-Peak Markers.

Observe the visualized signal and show/hide the orange ECG beat indicators.

STEP 39 Interact with Plotly waveform charts.

Zoom, pan, hover for time/amplitude, and inspect ECG above the optical pulse trace.

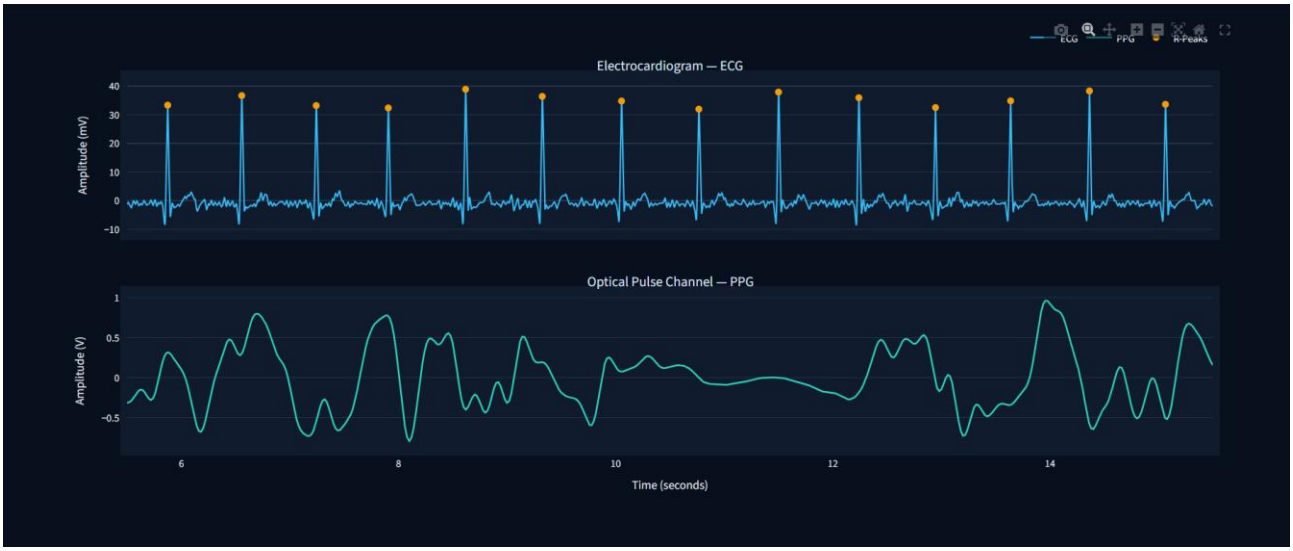


Figure 23. CardioTwin ECG waveform with R-peak markers and optical pulse trace.

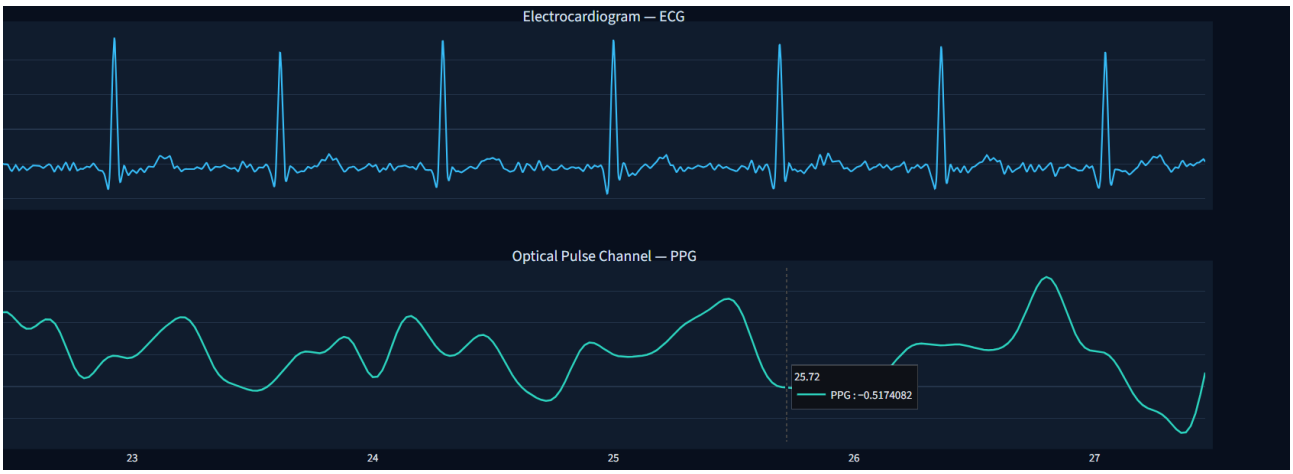


Figure 24. Navigation to the later part of the same recording (recorded replay, not live streaming).

10.3 Dashboard output behavior

The interface replays previously acquired AFE4950 measurements. It is not acquiring a second live ECG/PPG recording each time a slider is moved. In the reference demonstration, students reused the prior session rather than reconnecting a participant.

11 SK-AM62B electrical setup and serial console

11.1 Processor board connections

STEP 40 Place the SK-AM62B on an ESD-safe surface.

Inspect the USB-C power port, onboard boot-mode DIP switches, debug USB connection, and USB host interfaces before applying power.

STEP 41 Connect the compatible external USB-C PD adapter to the processor board.

During the reference setup the 65 W adapter was plugged into wall power; board LEDs illuminated.

STEP 42 Attach the separate debug micro-USB link to the Windows laptop.

Windows Device Manager enumerated four serial COM interfaces, COM5 through COM8.



Figure 25. TI SK-AM62B development board from the reference workbench.

```
>> Format-Table Status, Class, FriendlyName, InstanceId -AutoSize

Status Class     FriendlyName                                                 InstanceId
------ -----     ------------                                                 ----------
OK     USB       USB Root Hub (USB 3.0)                                       USB\ROOT_HUB30\4&8F3C413&0&0
OK     USB       USB Root Hub (USB 3.0)                                       USB\ROOT_HUB30\4&13BABFC3&0&0
OK     USB       USB Serial Converter A                                       USB\VID_0403&PID_6011&MI_00\6&326B9BB0&0&...
OK     USB       Intel(R) USB 3.20 eXtensible Host Controller - 1.20 (Microsoft) PCI\VEN_8086&DEV_7EC0&SUBSYS_8C4F103C&REV...
OK     USB       USB4(TM) Host Router (Microsoft)                             PCI\VEN_8086&DEV_7EC3&SUBSYS_8C4F103C&REV...
OK     DiskDrive WD PC SN560 SDDPNQE-1T00-2006                                SCSI\DISK&VEN_NVME&PROD_WD_PC_SN560_SDDP\...
OK     USB       USB Serial Converter B                                       USB\VID_0403&PID_6011&MI_01\6&326B9BB0&0&...
OK     USB       USB4(TM) Host Router (Microsoft)                             PCI\VEN_8086&DEV_7EC2&SUBSYS_8C4F103C&REV...
OK     USB       USB Serial Converter C                                       USB\VID_0403&PID_6011&MI_02\6&326B9BB0&0&...
OK     USB       USB4 Root Router (1.0)                                       USB4\ROOT_DEVICE_ROUTER&VID_8086&PID_7EC2...
OK     USB       USB4 Root Router (1.0)                                       USB4\ROOT_DEVICE_ROUTER&VID_8086&PID_7EC3...
OK     USB       USB Composite Device                                         USB\VID_0403&PID_6011\07231140042
OK     USB       USB Serial Converter D                                       USB\VID_0403&PID_6011&MI_03\6&326B9BB0&0&...
Error  USB       Unknown USB Device (Device Descriptor Request Failed)        USB\VID_0000&PID_0002\5&3ACE2079&0&1
OK     USB       Intel(R) USB 3.20 eXtensible Host Controller - 1.20 (Microsoft) PCI\VEN_8086&DEV_7E7D&SUBSYS_8C4F103C&REV...
OK     USB       USB Composite Device                                         USB\VID_0408&PID_5465\0001


(.venv) PS C:\Users\potai\Track1 Digital Twin>
```

Figure 26. Windows USB interface list, including serial converters A-D during processor diagnostics.

11.2 Open the U-Boot console

STEP 43 Start a serial terminal on COM5 at 115200 baud.

We used pyserial miniterm from a VS Code PowerShell terminal. COM number may differ by PC.

```
python -m serial.tools.miniterm COM5 115200
```

STEP 44 Press Enter and locate the => U-Boot prompt.

This confirms serial access to the board bootloader. The board responded to command input and printed the requested environment values.

STEP 45 Use version and mmc list for a non-destructive board check.

These commands inspect bootloader/build information and configured MMC controllers without modifying storage.

```
version
mmc list
```

12 USB DFU and U-Boot test sequence

12.1 Why USB DFU was used

A microSD card was not available during the processor bring-up. The test therefore used the USB DFU path with a host Windows computer and the TI AM62x U-Boot flash-writer files. This demonstrated that the board could accept bootloader-stage images and execute a diagnostic U-Boot command.

STEP 46 Configure the processor boot switches for the board-supported USB DFU boot mode.

Use the SK-AM62B hardware boot-switch table for the particular board revision. The exact SW1/SW2 positions were not retained in the captured lab record and are not guessed here.

STEP 47 Verify the DFU USB interface is recognized by Windows.

The tested DFU identity was USB VID:PID 0451:6165. The lab configured a compatible Windows USB driver for the DFU interface.

STEP 48 Install dfu-util 0.11 and obtain the TI U-Boot flash-writer files.

The tested boot files came from tools/DFU_flash/bin/am62xx-evm/hsfs in the TI uboot-flash-writer repository.

```
git clone https://git.ti.com/git/processor-sdk/uboot-flash-writer.git
```

STEP 49 Confirm that tiboot3.bin, tispl.bin, and u-boot.img exist.

In the reference experiment they were transferred in that order, using DFU alternate settings 0, 0, and 1 respectively.

```
$boot = "$env:USERPROFILE\Downloads\uboot-flash-writer\tools\DFU_flash\bin\am62xx-evm\hsfs"
# $dfu should point to your installed dfu-util.exe
& $dfu -d 0451:6165 -R -a 0 -D "$boot\tiboot3.bin"
& $dfu -d 0451:6165 -R -a 0 -D "$boot\tispl.bin"
& $dfu -d 0451:6165 -R -a 1 -D "$boot\u-boot.img"
```

STEP 50 Inspect the host-side download status.

The completed transfers were recorded as 333,163 bytes (tiboot3.bin), 960,044 bytes (tispl.bin), and 894,876 bytes (u-boot.img).

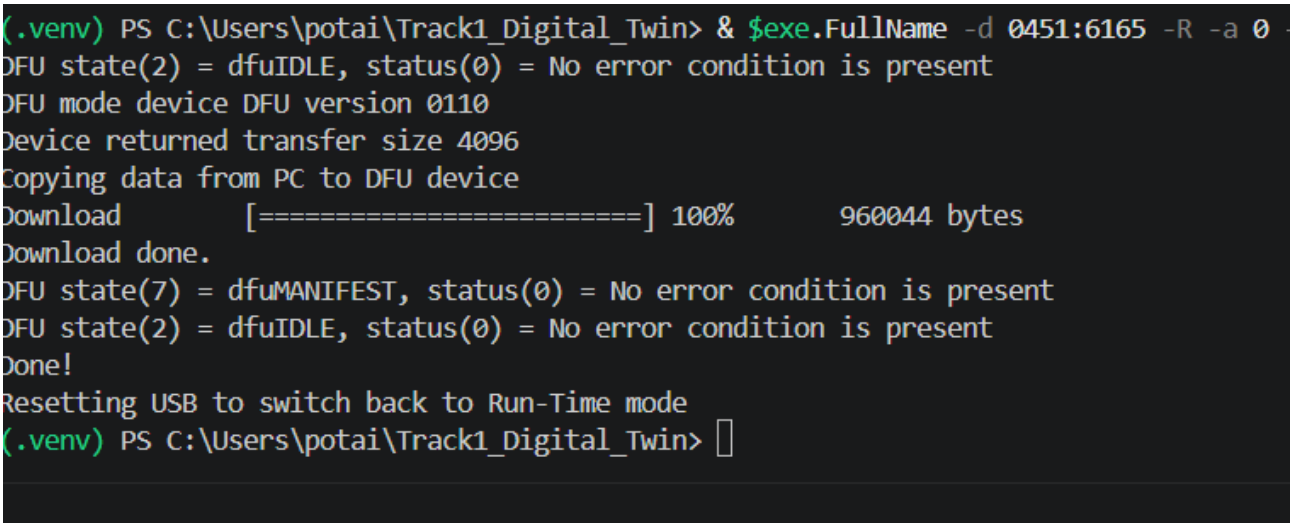


Figure 27. DFU transfer progress and successful completion during processor bootloader bring-up.

12.2 Execute a harmless diagnostic command

STEP 51 Prepare a simple uEnv.txt test message.

This test loaded a diagnostic echo command rather than Linux. It demonstrated U-Boot command execution.

```
"user_commands=echo TRACK1_UBOOT_READY" | Set-Content .\uEnv.txt -Encoding ascii
```

STEP 52 **Send the uEnv.txt over the U-Boot flash-writer DFU interface.**

The transfer succeeded and the U-Boot console printed the configured message after boot.

```
=> boot
## Info: input data size = 40815 = 0x9F6F
TRACK1_UBOOT_READY
=>
```

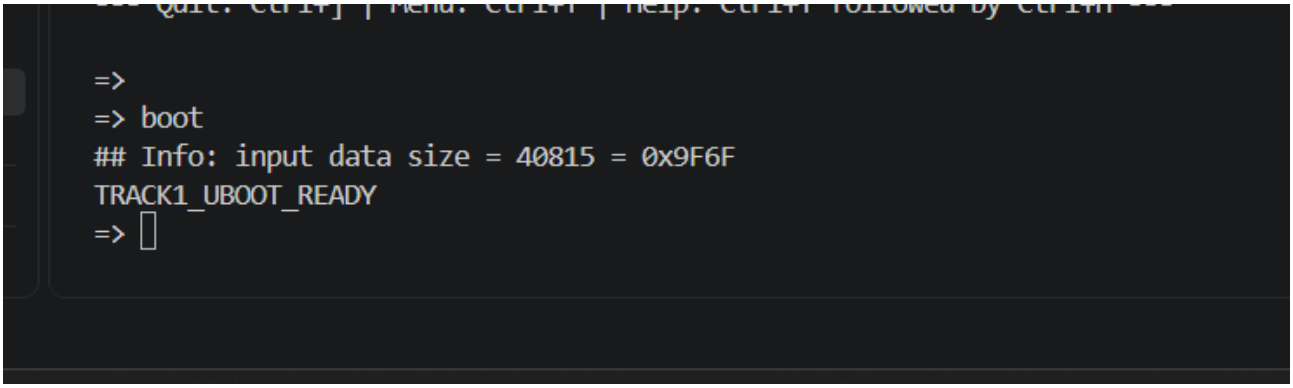


Figure 28. U-Boot console successfully executed the diagnostic TRACK1_UBOOT_READY message.

13 MMC, USB Mass Storage, and boot-state verification

13.1 Inspect memory-card controllers and partitions

STEP 53 **Inspect the MMC devices from the U-Boot prompt.**

The reference run listed mmc@fa10000: 0, mmc@fa00000: 1, and mmc@fa20000: 2.

STEP 54 **Select MMC device 0 and examine its partition table.**

The successful mmc dev 0 0 command and mmc part listing confirmed accessible user storage. This was not a Linux filesystem installation test.

```
=> mmc list
=> mmc dev 0 0
=> mmc part
=> ls mmc 0:1
=> ls mmc 0:2 /
```

13.2 Confirm the boot command currently installed

STEP 55 **Run printenv bootcmd at the => prompt.**

The reference output specified USB DFU reception of uEnv.txt followed by run user_commands, which explains the echoed test string.

```
=> printenv bootcmd
```

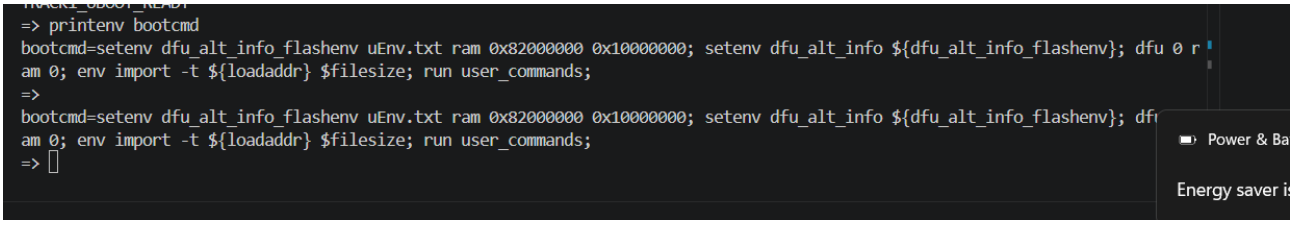


Figure 29. U-Boot bootcmd environment uses DFU to receive and execute uEnv.txt commands.

13.3 U-Boot USB Mass Storage check

STEP 56 Display the UMS command syntax and try exporting MMC device 0.

The serial console displayed UMS: LUN 0, dev 0, hwpart 0, confirming that the command began running.

```
=> help ums
=> ums 0 mmc 0
```

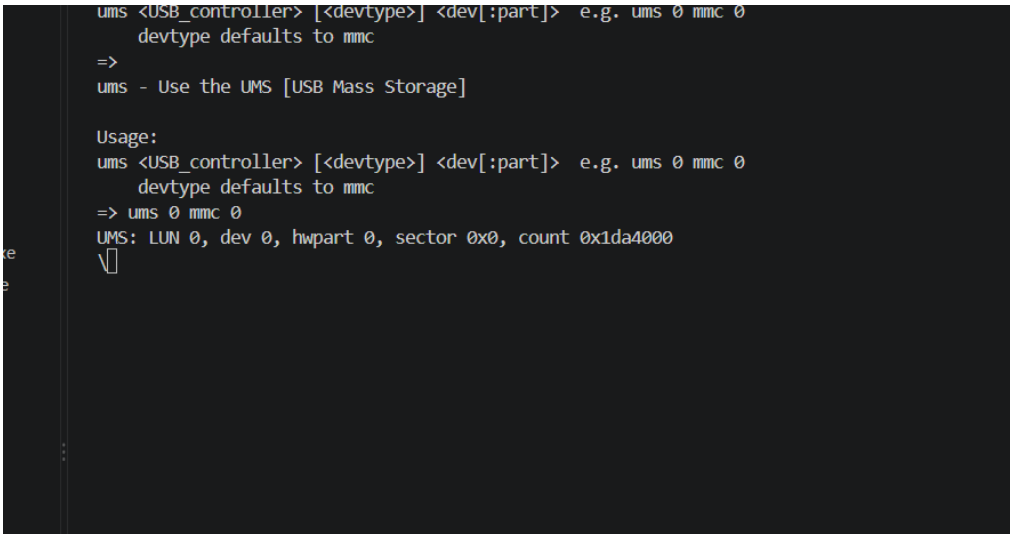


Figure 30. SK-AM62B entered U-Boot USB Mass Storage gadget mode.

STEP 57 Check Windows disk visibility without modifying disks.

The reference PowerShell listing showed only Disk 0, a laptop NVMe drive; no additional SK-AM62B USB disk was available for writing.

```
Get-Disk | Format-Table Number, FriendlyName, BusType, Size
```

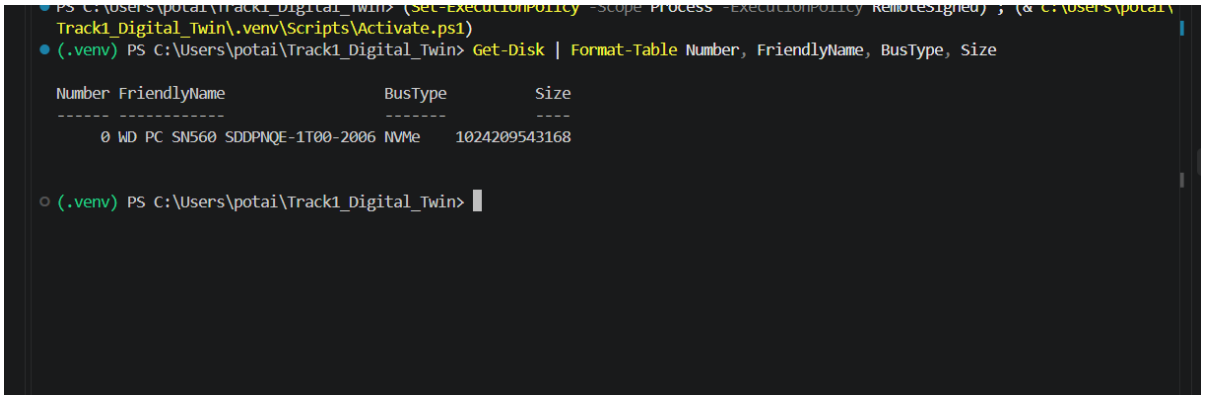


Figure 31. Windows listed the internal laptop SSD only; eMMC was not exposed as a host disk.

STEP 58 Inspect Windows USB device status when the storage disk is missing.

The captured system listing showed an Unknown USB Device with Device Descriptor Request Failed. Its precise association was not conclusively established.

```
Get-PnpDevice -PresentOnly | Where-Object {
  $_.Class -in @("USB", "DiskDrive") -or
  $_.FriendlyName -match "AM62|Mass Storage|USB Gadget"
} | Format-Table Status, Class, FriendlyName, InstanceId -AutoSize
```

STEP 59 Avoid unintended laptop or board flashing.

Do not initialize, format, or write the .wic image to Windows Disk 0. A TI Linux .wic disk image was downloaded to the laptop but not confirmed installed on SK-AM62B.

13.4 AFE4950-to-SK-AM62B: physical hookup

This is the wiring method for the two-board prototype. It uses the AFE4950EVM USB interface and the SK-AM62B USB-A HOST connector. The board photos and cable path are illustrated in the connection figure in Section 2.3. For a student who is replaying the supplied CSV, this USB link is optional and is not required to display the successful CardioTwin results.

STEP 60 Turn off both evaluation boards and remove body-connected electrodes.

Do not change cables with the participant connected; perform any human-connected measurement only with an instructor-approved, electrically isolated configuration.

STEP 61 Remove the AFE4950 micro-USB data cable from the PC.

The TI Bio-Sensing PC GUI cannot communicate with the AFE4950 through a cable that is instead connected to the SK-AM62B host. Save the recording CSV before switching.

STEP 62 Plug the Micro-USB end into the AFE4950EVM micro-USB connector.

Use a USB-A-to-Micro-USB cable that supports DATA, not a charge-only cable. Inspect the connector orientation.

STEP 63 Plug the USB-A end into the SK-AM62B USB-A HOST connector.

Use the rectangular Type-A host connector on the processor board. Do not use the SK-AM62B J11 USB-C power input for AFE data.

STEP 64 Provide power to the processor through J11 and retain the independent USB debug cable to the laptop.

The SK-AM62B uses an external USB-C PD adapter on J11; the serial-debug USB is a separate connection for console output.

STEP 65 Verify the AFE4950 powers up and inspect the SK-AM62B console.

The hardware should be secure and the TI sensor board status LED should illuminate. U-Boot will allow serial access, but does not constitute a running Linux data-acquisition service.

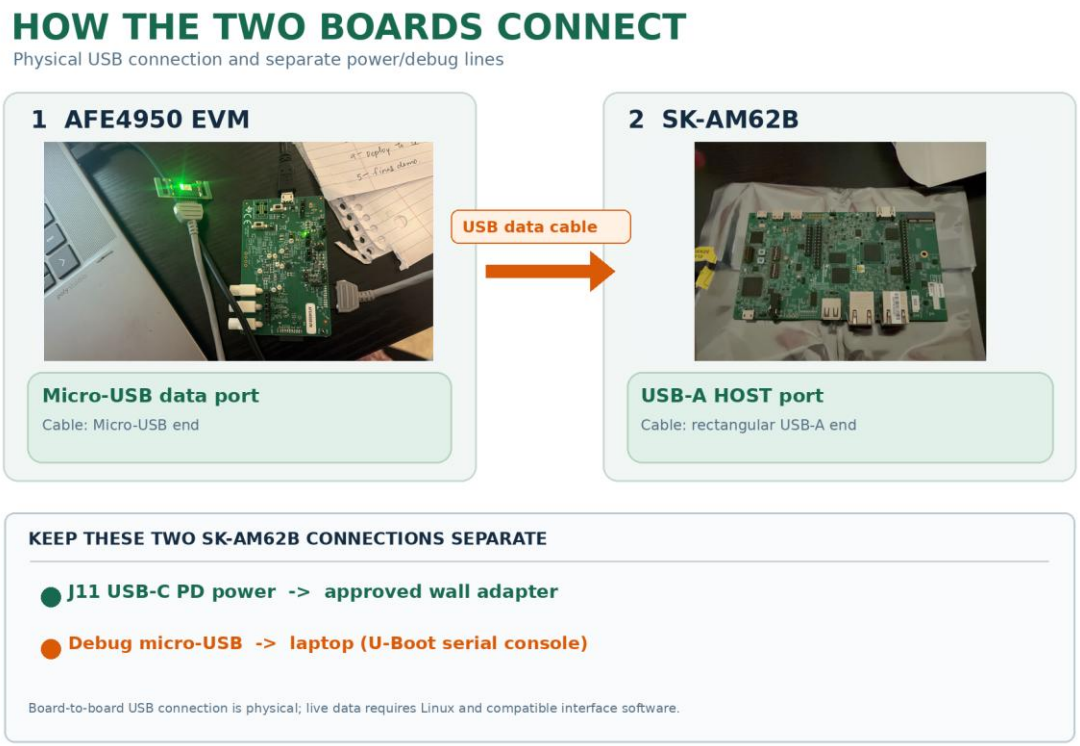


Figure 32. Connection recap: AFE4950 Micro-USB to SK-AM62B USB-A host, J11 power supply, separate debug micro-USB.

13.5 Verify the USB sensor interface under Linux (extension stage)

Once a suitable Linux image has actually been installed and booted on the SK-AM62B, the following NON-DESTRUCTIVE commands are an appropriate first check. They were not run successfully in our U-Boot-only session, and therefore are presented as a subsequent lab verification procedure.

STEP 66 Boot a compatible Linux installation and access the board shell.

Use the current board-revision-matched TI Processor SDK instructions. Do not treat a downloaded .wic file on Windows as an installed operating system.

STEP 67 Inspect USB enumeration and kernel messages for the AFE4950 connection.

Run lsusb and dmesg. CDC interfaces might produce /dev/ttyACM* devices, depending on the firmware and USB driver.

```
lsusb
dmesg | tail -n 40
ls -l /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
```

STEP 68 Implement the AFE4950 command/data communication in processor-side software.

USB enumeration confirms device presence, not physiological streaming. The EVM control protocol and matching host software must be supported or ported before claiming an onboard ECG/PPG-processing result.

Appendix A Command reference

A.1 Host Python analysis and dashboard

```
cd C:\Users\potai\Track1_Digital_Twin
.\.venv\Scripts\Activate.ps1
python analyze_signals.py
```

```
python -m streamlit run dashboard.py
```

A.2 SK-AM62B serial and non-destructive checks

```
python -m serial.tools.miniterm COM5 115200
# In the board console:
=> version
=> mmc list
=> mmc dev 0 0
=> mmc part
=> printenv bootcmd
=> help ums
=> ums 0 mmc 0
```

A.3 Windows device inspection

```
Get-Disk | Format-Table Number, FriendlyName, BusType, Size
Get-PnpDevice -PresentOnly | Where-Object {
  $_.Class -in @("USB", "DiskDrive")
} | Format-Table Status, Class, FriendlyName, InstanceId -AutoSize
```

A.4 Proven result summary

```
ECG beat candidates:       39
Optical peak candidates:    37
ECG / optical matches:      21
Estimated heart rate:       84.99 BPM
Record duration:            27.48 s
ECG sampling:               500 Hz
Optical sampling:            50 Hz
```

Appendix B Troubleshooting and common checks

SYMPTOM	USEFUL CHECK
TI GUI does not see AFE4950	Confirm USB cable supports data; check CDC driver and TI GUI installation
AFE firmware loader says no device	Check S1 bootloader entry sequence and matching firmware; follow the TI guide
Signal plot appears empty	Review channel selection, register import, capture start, and correct connection
Python cannot find CSV	Place CSV in data folder; use script-relative Path object
Unexpected CSV values / swapped time columns	Read file with pandas index_col=False because of trailing delimiter
PowerShell says boot is unknown	Enter boot only at => inside the serial terminal
No SK-AM62B serial text	Check debug USB cable, COM number, 115200 baud, and bootloader prompt
Windows Get-Disk shows only Disk 0	Do not flash; identify and establish an actual target disk first
Laptop reports AC adapter incompatible	Disconnect the suspect USB-C data path and restore the normal laptop charger
Streamlit prints use_container_width warning	Deprecation message; dashboard can still operate

Appendix C File inventory and reference documents

C.1 Prototype file map

FILE	ROLE
data/AFE4950_CAPTURED_DATA.csv	Original TI acquisition export used throughout the example
analyze_signals.py	ECG/optical processing and metric generation
dashboard.py	Streamlit CardioTwin application
data/metrics.json	Stored analysis summary
data/candidate_matches.csv	ECG/optical candidate pair records
tisdk-default-image-am62xx-evm-12.01.00.05.03.rootfs.wic	TI Linux image downloaded to the Windows host (not deployed)
tiboot3.bin / tispl.bin / u-boot.img	TI AM62x DFU bootloader-stage test artifacts
uEnv.txt	DFU diagnostic script used to echo TRACK1_UBOOT_READY

C.2 Source references

Texas Instruments. AFE4950 Evaluation Module Development Guide, SLAU832, June 2020, especially sections 1-6 (evaluation overview, software setup, connector interface, register generation, and data export). User-supplied laboratory photographs, videos, CSV, prototype screenshots, and console logs from October 24-25, 2026.

See Texas Instruments AM62x SK Evaluation Modules User's Guide (SPRUJ40, board-revision-appropriate edition) for the Type-A USB host, USB-C J11 power, debug UART connections, boot switches, and board-to-sensor USB-host verification. Exact Linux/eMMC procedures depend on the board revision and SDK release.